



向线程池提交任务（上）

北京理工大学计算机学院
金旭亮

如何提交任务给线程池?



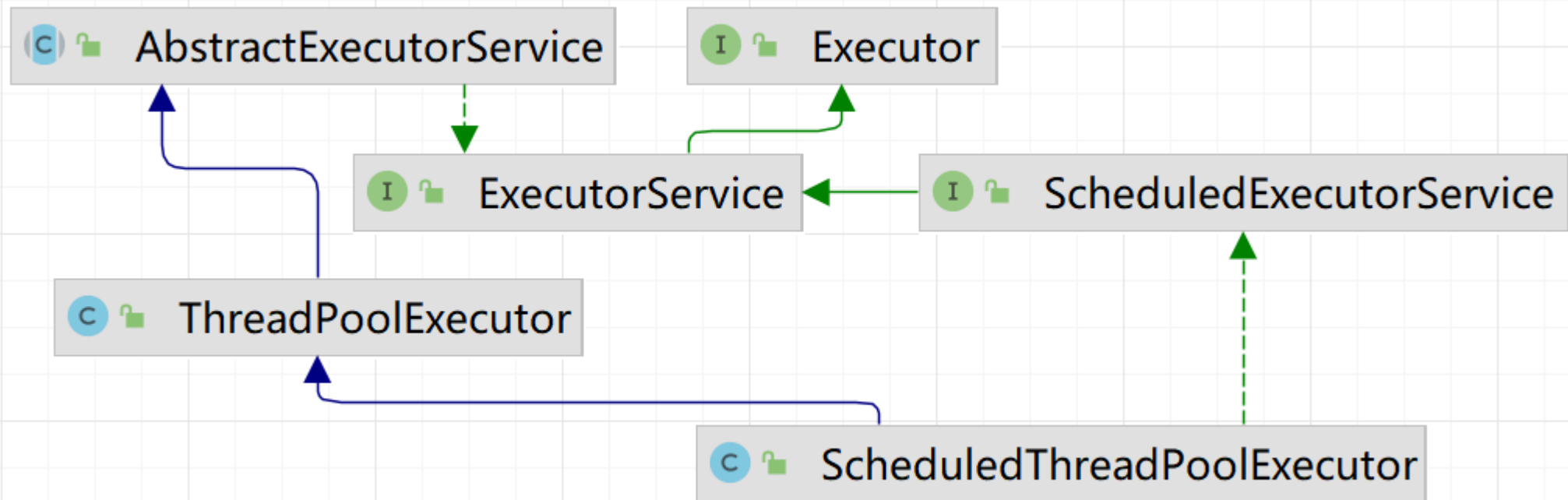
JDK中定义了一个Executor接口:

```
public interface Executor {  
    void execute(Runnable command);  
}
```

上述接口定义了一个execute()方法, 它接收一个Runnable对象。

线程池对象实现了上述接口, 因此, 只需要将任务封装为Runnable对象, 就可调用线程池对象的execute()方法执行它。

JDK中线程池相关的类型



Executors

  newFixedThreadPool (int, ThreadFactory)	ExecutorService
  newScheduledThreadPool (int, ThreadFactory)	ScheduledExecutorService
  newSingleThreadScheduledExecutor (ThreadFactory)	ScheduledExecutorService
  newCachedThreadPool (ThreadFactory)	ExecutorService
  callable (Runnable , T)	Callable < T >
  newWorkStealingPool ()	ExecutorService
  unconfigurableScheduledExecutorService (ScheduledExecutorService)	ScheduledExecutorService
  privilegedCallable (Callable < T >)	Callable < T >
  newSingleThreadExecutor (ThreadFactory)	ExecutorService
  newCachedThreadPool ()	ExecutorService
  newSingleThreadExecutor ()	ExecutorService
  newWorkStealingPool (int)	ExecutorService
  callable (PrivilegedAction < ? >)	Callable < Object >
  privilegedThreadFactory ()	ThreadFactory
  newScheduledThreadPool (int)	ScheduledExecutorService
  defaultThreadFactory ()	ThreadFactory
  callable (Runnable)	Callable < Object >
  callable (PrivilegedExceptionAction < ? >)	Callable < Object >
  privilegedCallableUsingCurrentClassLoader (Callable < T >)	Callable < T >
  newSingleThreadScheduledExecutor ()	ScheduledExecutorService
  newFixedThreadPool (int)	ExecutorService
  unconfigurableExecutorService (ExecutorService)	ExecutorService

线程池的功能由相关接口进行抽象，它的实例，则由 Executors 工具类负责。

使用线程池执行任务

```
private static void useThreadPool() {  
    //创建一个包容两个线程的线程池  
    var executor :ExecutorService = Executors.newFixedThreadPool( nThreads: 2);  
    //输出: java.util.concurrent.ThreadPoolExecutor  
    System.out.println(executor.getClass().getName());  
  
    try {  
        for(int i=0;i<5;i++){  
            //如果向线程池提交多个任务，可以看到线程对象得到了“重用”  
            executor.execute(() -> {  
                //pool-1-thread-1或pool-1-thread-2  
                System.out.println(Thread.currentThread().getName());  
            });  
        }  
    } finally {  
        //线程池启动之后，会一直运行，如果不显式关闭它，会导致本进程一直无法结束。  
        executor.shutdown();  
    }  
}
```

UseRunnable.java

可以看到，只要将要执行的任务用Runnable进行封装，调用execute()方法就能将它放到线程池的任务队列中。

线程池中的两个线程，反复执行任务队列中的任务。

使用Runnable提交任务的特点



没有返回值



无法抛出Checked Exception，因此，它里面的代码，如果抛出了异常，只能try/catch。



为了弥补以上两个缺陷，推出了Callable接口，下一讲介绍。